

Smart Contract Templates

The Ultimate Web3 & TRON Solidity Library

White Solutions

Published: September 2026

Copyright © 2026 White Solutions

All rights reserved.

This document is licensed for commercial use by the original
purchaser.

Retail Price: \$929.00 USD

Contents

Foreword	v
1 Introduction to TRON Smart Contracts	1
1.1 The TRC-20 Token Standard	1
2 Core Interfaces	3
2.1 ITRC20 Core Interface Part 1	3
2.2 ITRC20 Core Interface Part 2	4
2.3 ITRC20 Core Interface Part 3	5
2.4 ITRC20 Core Interface Part 4	6
2.5 ITRC20 Core Interface Part 5	7
2.6 ITRC20 Core Interface Part 6	8
2.7 ITRC20 Core Interface Part 7	9
2.8 ITRC20 Core Interface Part 8	10
2.9 ITRC20 Core Interface Part 9	11
2.10 ITRC20 Core Interface Part 10	12
3 Advanced TRC-20 Token Implementations	13
3.1 Deflationary Token Model 1	13
3.1.1 Energy Optimization Strategy 1	14
3.2 Deflationary Token Model 2	15
3.2.1 Energy Optimization Strategy 2	16
3.3 Deflationary Token Model 3	17
3.3.1 Energy Optimization Strategy 3	18

3.4	Deflationary Token Model 4	19
3.4.1	Energy Optimization Strategy 4	20
3.5	Deflationary Token Model 5	21
3.5.1	Energy Optimization Strategy 5	22
3.6	Deflationary Token Model 6	23
3.6.1	Energy Optimization Strategy 6	24
3.7	Deflationary Token Model 7	25
3.7.1	Energy Optimization Strategy 7	26
3.8	Deflationary Token Model 8	27
3.8.1	Energy Optimization Strategy 8	28
3.9	Deflationary Token Model 9	29
3.9.1	Energy Optimization Strategy 9	30
3.10	Deflationary Token Model 10	31
3.10.1	Energy Optimization Strategy 10	32
3.11	Deflationary Token Model 11	33
3.11.1	Energy Optimization Strategy 11	34
3.12	Deflationary Token Model 12	35
3.12.1	Energy Optimization Strategy 12	36
3.13	Deflationary Token Model 13	37
3.13.1	Energy Optimization Strategy 13	38
3.14	Deflationary Token Model 14	39
3.14.1	Energy Optimization Strategy 14	40
3.15	Deflationary Token Model 15	41
3.15.1	Energy Optimization Strategy 15	42
3.16	Deflationary Token Model 16	43
3.16.1	Energy Optimization Strategy 16	44
3.17	Deflationary Token Model 17	45
3.17.1	Energy Optimization Strategy 17	46
3.18	Deflationary Token Model 18	47
3.18.1	Energy Optimization Strategy 18	48
3.19	Deflationary Token Model 19	49
3.19.1	Energy Optimization Strategy 19	50

3.20 Deflationary Token Model 20	51
3.20.1 Energy Optimization Strategy 20	52
3.21 Deflationary Token Model 21	53
3.21.1 Energy Optimization Strategy 21	54
3.22 Deflationary Token Model 22	55
3.22.1 Energy Optimization Strategy 22	56
3.23 Deflationary Token Model 23	57
3.23.1 Energy Optimization Strategy 23	58
3.24 Deflationary Token Model 24	59
3.24.1 Energy Optimization Strategy 24	60
3.25 Deflationary Token Model 25	61
3.25.1 Energy Optimization Strategy 25	62
3.26 Deflationary Token Model 26	63
3.26.1 Energy Optimization Strategy 26	64
3.27 Deflationary Token Model 27	65
3.27.1 Energy Optimization Strategy 27	66
3.28 Deflationary Token Model 28	67
3.28.1 Energy Optimization Strategy 28	68
3.29 Deflationary Token Model 29	69
3.29.1 Energy Optimization Strategy 29	70
3.30 Deflationary Token Model 30	71
3.30.1 Energy Optimization Strategy 30	72
3.31 Deflationary Token Model 31	73
3.31.1 Energy Optimization Strategy 31	74
3.32 Deflationary Token Model 32	75
3.32.1 Energy Optimization Strategy 32	76
3.33 Deflationary Token Model 33	77
3.33.1 Energy Optimization Strategy 33	78
3.34 Deflationary Token Model 34	79
3.34.1 Energy Optimization Strategy 34	80
3.35 Deflationary Token Model 35	81
3.35.1 Energy Optimization Strategy 35	82

3.36	Deflationary Token Model 36	83
3.36.1	Energy Optimization Strategy 36	84
3.37	Deflationary Token Model 37	85
3.37.1	Energy Optimization Strategy 37	86
3.38	Deflationary Token Model 38	87
3.38.1	Energy Optimization Strategy 38	88
3.39	Deflationary Token Model 39	89
3.39.1	Energy Optimization Strategy 39	90
3.40	Deflationary Token Model 40	91
3.40.1	Energy Optimization Strategy 40	92
3.41	Deflationary Token Model 41	93
3.41.1	Energy Optimization Strategy 41	94
3.42	Deflationary Token Model 42	95
3.42.1	Energy Optimization Strategy 42	96
3.43	Deflationary Token Model 43	97
3.43.1	Energy Optimization Strategy 43	98
3.44	Deflationary Token Model 44	99
3.44.1	Energy Optimization Strategy 44	100
3.45	Deflationary Token Model 45	101
3.45.1	Energy Optimization Strategy 45	102
3.46	Deflationary Token Model 46	103
3.46.1	Energy Optimization Strategy 46	104
3.47	Deflationary Token Model 47	105
3.47.1	Energy Optimization Strategy 47	106
3.48	Deflationary Token Model 48	107
3.48.1	Energy Optimization Strategy 48	108
3.49	Deflationary Token Model 49	109
3.49.1	Energy Optimization Strategy 49	110
3.50	Deflationary Token Model 50	111
3.50.1	Energy Optimization Strategy 50	112

4	Staking and Yield Farming Protocols	113
4.1	Staking Vault Boilerplate 1	113
4.1.1	Yield Analysis 1	114
4.2	Staking Vault Boilerplate 2	115
4.2.1	Yield Analysis 2	115
4.3	Staking Vault Boilerplate 3	116
4.3.1	Yield Analysis 3	116
4.4	Staking Vault Boilerplate 4	117
4.4.1	Yield Analysis 4	117
4.5	Staking Vault Boilerplate 5	118
4.5.1	Yield Analysis 5	118
4.6	Staking Vault Boilerplate 6	119
4.6.1	Yield Analysis 6	119
4.7	Staking Vault Boilerplate 7	120
4.7.1	Yield Analysis 7	120
4.8	Staking Vault Boilerplate 8	121
4.8.1	Yield Analysis 8	121
4.9	Staking Vault Boilerplate 9	122
4.9.1	Yield Analysis 9	122
4.10	Staking Vault Boilerplate 10	123
4.10.1	Yield Analysis 10	123
4.11	Staking Vault Boilerplate 11	124
4.11.1	Yield Analysis 11	124
4.12	Staking Vault Boilerplate 12	125
4.12.1	Yield Analysis 12	125
4.13	Staking Vault Boilerplate 13	126
4.13.1	Yield Analysis 13	126
4.14	Staking Vault Boilerplate 14	127
4.14.1	Yield Analysis 14	127
4.15	Staking Vault Boilerplate 15	128
4.15.1	Yield Analysis 15	128
4.16	Staking Vault Boilerplate 16	129

4.16.1 Yield Analysis 16	129
4.17 Staking Vault Boilerplate 17	130
4.17.1 Yield Analysis 17	130
4.18 Staking Vault Boilerplate 18	131
4.18.1 Yield Analysis 18	131
4.19 Staking Vault Boilerplate 19	132
4.19.1 Yield Analysis 19	132
4.20 Staking Vault Boilerplate 20	133
4.20.1 Yield Analysis 20	133
4.21 Staking Vault Boilerplate 21	134
4.21.1 Yield Analysis 21	134
4.22 Staking Vault Boilerplate 22	135
4.22.1 Yield Analysis 22	135
4.23 Staking Vault Boilerplate 23	136
4.23.1 Yield Analysis 23	136
4.24 Staking Vault Boilerplate 24	137
4.24.1 Yield Analysis 24	137
4.25 Staking Vault Boilerplate 25	138
4.25.1 Yield Analysis 25	138
4.26 Staking Vault Boilerplate 26	139
4.26.1 Yield Analysis 26	139
4.27 Staking Vault Boilerplate 27	140
4.27.1 Yield Analysis 27	140
4.28 Staking Vault Boilerplate 28	141
4.28.1 Yield Analysis 28	141
4.29 Staking Vault Boilerplate 29	142
4.29.1 Yield Analysis 29	142
4.30 Staking Vault Boilerplate 30	143
4.30.1 Yield Analysis 30	143
4.31 Staking Vault Boilerplate 31	144
4.31.1 Yield Analysis 31	144
4.32 Staking Vault Boilerplate 32	145

4.32.1	Yield Analysis 32	145
4.33	Staking Vault Boilerplate 33	146
4.33.1	Yield Analysis 33	146
4.34	Staking Vault Boilerplate 34	147
4.34.1	Yield Analysis 34	147
4.35	Staking Vault Boilerplate 35	148
4.35.1	Yield Analysis 35	148
4.36	Staking Vault Boilerplate 36	149
4.36.1	Yield Analysis 36	149
4.37	Staking Vault Boilerplate 37	150
4.37.1	Yield Analysis 37	150
4.38	Staking Vault Boilerplate 38	151
4.38.1	Yield Analysis 38	151
4.39	Staking Vault Boilerplate 39	152
4.39.1	Yield Analysis 39	152
4.40	Staking Vault Boilerplate 40	153
4.40.1	Yield Analysis 40	153
5	Governance and Multi-Signature Wallets	155
5.1	Multi-Sig Authority Contract 1	155
5.1.1	Security Audit Check 1	156
5.2	Multi-Sig Authority Contract 2	157
5.2.1	Security Audit Check 2	158
5.3	Multi-Sig Authority Contract 3	159
5.3.1	Security Audit Check 3	160
5.4	Multi-Sig Authority Contract 4	161
5.4.1	Security Audit Check 4	162
5.5	Multi-Sig Authority Contract 5	163
5.5.1	Security Audit Check 5	164
5.6	Multi-Sig Authority Contract 6	165
5.6.1	Security Audit Check 6	166
5.7	Multi-Sig Authority Contract 7	167
5.7.1	Security Audit Check 7	168

5.8	Multi-Sig Authority Contract 8	169
5.8.1	Security Audit Check 8	170
5.9	Multi-Sig Authority Contract 9	171
5.9.1	Security Audit Check 9	172
5.10	Multi-Sig Authority Contract 10	173
5.10.1	Security Audit Check 10	174
5.11	Multi-Sig Authority Contract 11	175
5.11.1	Security Audit Check 11	176
5.12	Multi-Sig Authority Contract 12	177
5.12.1	Security Audit Check 12	178
5.13	Multi-Sig Authority Contract 13	179
5.13.1	Security Audit Check 13	180
5.14	Multi-Sig Authority Contract 14	181
5.14.1	Security Audit Check 14	182
5.15	Multi-Sig Authority Contract 15	183
5.15.1	Security Audit Check 15	184
5.16	Multi-Sig Authority Contract 16	185
5.16.1	Security Audit Check 16	186
5.17	Multi-Sig Authority Contract 17	187
5.17.1	Security Audit Check 17	188
5.18	Multi-Sig Authority Contract 18	189
5.18.1	Security Audit Check 18	190
5.19	Multi-Sig Authority Contract 19	191
5.19.1	Security Audit Check 19	192
5.20	Multi-Sig Authority Contract 20	193
5.20.1	Security Audit Check 20	194
5.21	Multi-Sig Authority Contract 21	195
5.21.1	Security Audit Check 21	196
5.22	Multi-Sig Authority Contract 22	197
5.22.1	Security Audit Check 22	198
5.23	Multi-Sig Authority Contract 23	199
5.23.1	Security Audit Check 23	200

5.24 Multi-Sig Authority Contract 24	201
5.24.1 Security Audit Check 24	202
5.25 Multi-Sig Authority Contract 25	203
5.25.1 Security Audit Check 25	204
5.26 Multi-Sig Authority Contract 26	205
5.26.1 Security Audit Check 26	206
5.27 Multi-Sig Authority Contract 27	207
5.27.1 Security Audit Check 27	208
5.28 Multi-Sig Authority Contract 28	209
5.28.1 Security Audit Check 28	210
5.29 Multi-Sig Authority Contract 29	211
5.29.1 Security Audit Check 29	212
5.30 Multi-Sig Authority Contract 30	213
5.30.1 Security Audit Check 30	214
6 Conclusion	215

Foreword

In the rapidly evolving landscape of decentralized finance, deploying secure, scalable, and optimized smart contracts is paramount. **White Solutions** has engineered this massive \$929 premium library to provide you with enterprise-grade TRON and EVM-compatible Solidity templates. Whether you are launching a TRC-20 token, a DeFi staking protocol, or automated liquidity pools, this guide provides the exact boilerplate code required.

Chapter 1

Introduction to TRON Smart Contracts

1.1 The TRC-20 Token Standard

TRC-20 is a technical standard used for smart contracts on the TRON blockchain for implementing tokens. It is fully compatible with Ethereum's ERC-20. Due to TRON's Delegated Proof-of-Stake (DPoS) consensus, transactions are confirmed in seconds with near-zero fees when energy and bandwidth are properly managed.

Chapter 2

Core Interfaces

2.1 ITRC20 Core Interface Part 1

```
1 // Interface Definition Block 1
2 interface ITRC20_1 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.1: ITRC20 Core Interface 1

2.2 ITRC20 Core Interface Part 2

```
1 // Interface Definition Block 2
2 interface ITRC20_2 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.2: ITRC20 Core Interface 2

2.3 ITRC20 Core Interface Part 3

```
1 // Interface Definition Block 3
2 interface ITRC20_3 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.3: ITRC20 Core Interface 3

2.4 ITRC20 Core Interface Part 4

```
1 // Interface Definition Block 4
2 interface ITRC20_4 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.4: ITRC20 Core Interface 4

2.5 ITRC20 Core Interface Part 5

```
1 // Interface Definition Block 5
2 interface ITRC20_5 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.5: ITRC20 Core Interface 5

2.6 ITRC20 Core Interface Part 6

```
1 // Interface Definition Block 6
2 interface ITRC20_6 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.6: ITRC20 Core Interface 6

2.7 ITRC20 Core Interface Part 7

```
1 // Interface Definition Block 7
2 interface ITRC20_7 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.7: ITRC20 Core Interface 7

2.8 ITRC20 Core Interface Part 8

```
1 // Interface Definition Block 8
2 interface ITRC20_8 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.8: ITRC20 Core Interface 8

2.9 ITRC20 Core Interface Part 9

```
1 // Interface Definition Block 9
2 interface ITRC20_9 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.9: ITRC20 Core Interface 9

2.10 ITRC20 Core Interface Part 10

```
1 // Interface Definition Block 10
2 interface ITRC20_10 {
3     function totalSupply() external view returns (uint256);
4     function balanceOf(address account) external view returns (
5         uint256);
6     function transfer(address recipient, uint256 amount) external
7         returns (bool);
8     function allowance(address owner, address spender) external
9         view returns (uint256);
10    function approve(address spender, uint256 amount) external
11        returns (bool);
12    function transferFrom(address sender, address recipient,
13        uint256 amount) external returns (bool);
14    event Transfer(address indexed from, address indexed to,
15        uint256 value);
16    event Approval(address indexed owner, address indexed spender
17        , uint256 value);
18 }
```

Listing 2.10: ITRC20 Core Interface 10

Chapter 3

Advanced TRC-20 Token Implementations

3.1 Deflationary Token Model 1

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract DeflationaryToken1 {
5     string public name = "WhiteSolutions Deflationary";
6     string public symbol = "WSD1";
7     uint8 public decimals = 6;
8     uint256 public totalSupply;
9
10    mapping(address => uint256) public balanceOf;
11    mapping(address => mapping(address => uint256)) public allowance;
12
13    uint256 public burnRate = 2; // 2% burn on transfer
14
15    constructor(uint256 _initialSupply) {
16        totalSupply = _initialSupply * 10 ** uint256(decimals);
17        balanceOf[msg.sender] = totalSupply;
18    }
19
20    function _transfer(address sender, address recipient, uint256 amount) internal {
21        require(sender != address(0), "Transfer from zero address
```

```

22     ");
23     require(recipient != address(0), "Transfer to zero
24     address");
25
26     uint256 burnAmount = (amount * burnRate) / 100;
27     uint256 sendAmount = amount - burnAmount;
28
29     balanceOf[sender] -= amount;
30     balanceOf[recipient] += sendAmount;
31     totalSupply -= burnAmount;
32 }
33
34 // Transfer logic hooks into _transfer
35 function transfer(address recipient, uint256 amount) external
36     returns (bool) {
37     _transfer(msg.sender, recipient, amount);
38     return true;
39 }

```

Listing 3.1: Deflationary TRC-20 Implementation Module 1

3.1.1 Energy Optimization Strategy 1

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.2 Deflationary Token Model 2

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken2 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD2";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
21         amount) internal {
22         require(sender != address(0), "Transfer from zero address
23         ");
24         require(recipient != address(0), "Transfer to zero
25         address");
26
27         uint256 burnAmount = (amount * burnRate) / 100;
28         uint256 sendAmount = amount - burnAmount;
29
30         balanceOf[sender] -= amount;
31         balanceOf[recipient] += sendAmount;
32         totalSupply -= burnAmount;
33     }
34
35     // Transfer logic hooks into _transfer
36     function transfer(address recipient, uint256 amount) external
37         returns (bool) {
38         _transfer(msg.sender, recipient, amount);
39         return true;
40     }
41 }
```

Listing 3.2: Deflationary TRC-20 Implementation Module 2

3.2.1 Energy Optimization Strategy 2

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.3 Deflationary Token Model 3

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken3 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD3";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
21         amount) internal {
22         require(sender != address(0), "Transfer from zero address
23         ");
24         require(recipient != address(0), "Transfer to zero
25         address");
26
27         uint256 burnAmount = (amount * burnRate) / 100;
28         uint256 sendAmount = amount - burnAmount;
29
30         balanceOf[sender] -= amount;
31         balanceOf[recipient] += sendAmount;
32         totalSupply -= burnAmount;
33     }
34
35     // Transfer logic hooks into _transfer
36     function transfer(address recipient, uint256 amount) external
37         returns (bool) {
38         _transfer(msg.sender, recipient, amount);
39         return true;
40     }
41 }
```

Listing 3.3: Deflationary TRC-20 Implementation Module 3

3.3.1 Energy Optimization Strategy 3

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.4 Deflationary Token Model 4

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken4 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD4";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

Listing 3.4: Deflationary TRC-20 Implementation Module 4

3.4.1 Energy Optimization Strategy 4

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.5 Deflationary Token Model 5

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken5 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD5";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

Listing 3.5: Deflationary TRC-20 Implementation Module 5

3.5.1 Energy Optimization Strategy 5

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.6 Deflationary Token Model 6

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken6 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD6";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
21         amount) internal {
22         require(sender != address(0), "Transfer from zero address
23         ");
24         require(recipient != address(0), "Transfer to zero
25         address");
26
27         uint256 burnAmount = (amount * burnRate) / 100;
28         uint256 sendAmount = amount - burnAmount;
29
30         balanceOf[sender] -= amount;
31         balanceOf[recipient] += sendAmount;
32         totalSupply -= burnAmount;
33     }
34
35     // Transfer logic hooks into _transfer
36     function transfer(address recipient, uint256 amount) external
37         returns (bool) {
38         _transfer(msg.sender, recipient, amount);
39         return true;
40     }
41 }
```

Listing 3.6: Deflationary TRC-20 Implementation Module 6

3.6.1 Energy Optimization Strategy 6

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.7 Deflationary Token Model 7

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken7 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD7";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

Listing 3.7: Deflationary TRC-20 Implementation Module 7

3.7.1 Energy Optimization Strategy 7

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.8 Deflationary Token Model 8

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken8 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD8";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

Listing 3.8: Deflationary TRC-20 Implementation Module 8

3.8.1 Energy Optimization Strategy 8

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.9 Deflationary Token Model 9

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken9 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD9";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
21         amount) internal {
22         require(sender != address(0), "Transfer from zero address
23         ");
24         require(recipient != address(0), "Transfer to zero
25         address");
26
27         uint256 burnAmount = (amount * burnRate) / 100;
28         uint256 sendAmount = amount - burnAmount;
29
30         balanceOf[sender] -= amount;
31         balanceOf[recipient] += sendAmount;
32         totalSupply -= burnAmount;
33     }
34
35     // Transfer logic hooks into _transfer
36     function transfer(address recipient, uint256 amount) external
37         returns (bool) {
38         _transfer(msg.sender, recipient, amount);
39         return true;
40     }
41 }
```

37 }

Listing 3.9: Deflationary TRC-20 Implementation Module 9

3.9.1 Energy Optimization Strategy 9

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.10 Deflationary Token Model 10

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken10 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD10";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.10: Deflationary TRC-20 Implementation Module 10

3.10.1 Energy Optimization Strategy 10

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.11 Deflationary Token Model 11

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken11 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD11";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.11: Deflationary TRC-20 Implementation Module 11

3.11.1 Energy Optimization Strategy 11

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.12 Deflationary Token Model 12

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken12 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD12";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.12: Deflationary TRC-20 Implementation Module 12

3.12.1 Energy Optimization Strategy 12

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.13 Deflationary Token Model 13

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken13 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD13";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.13: Deflationary TRC-20 Implementation Module 13

3.13.1 Energy Optimization Strategy 13

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.14 Deflationary Token Model 14

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken14 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD14";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.14: Deflationary TRC-20 Implementation Module 14

3.14.1 Energy Optimization Strategy 14

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.15 Deflationary Token Model 15

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken15 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD15";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.15: Deflationary TRC-20 Implementation Module 15

3.15.1 Energy Optimization Strategy 15

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.16 Deflationary Token Model 16

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken16 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD16";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.16: Deflationary TRC-20 Implementation Module 16

3.16.1 Energy Optimization Strategy 16

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.17 Deflationary Token Model 17

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken17 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD17";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.17: Deflationary TRC-20 Implementation Module 17

3.17.1 Energy Optimization Strategy 17

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.18 Deflationary Token Model 18

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken18 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD18";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.18: Deflationary TRC-20 Implementation Module 18

3.18.1 Energy Optimization Strategy 18

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.19 Deflationary Token Model 19

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken19 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD19";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.19: Deflationary TRC-20 Implementation Module 19

3.19.1 Energy Optimization Strategy 19

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.20 Deflationary Token Model 20

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken20 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD20";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.20: Deflationary TRC-20 Implementation Module 20

3.20.1 Energy Optimization Strategy 20

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.21 Deflationary Token Model 21

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken21 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD21";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256 amount) internal {
21         require(sender != address(0), "Transfer from zero address");
22         require(recipient != address(0), "Transfer to zero address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.21: Deflationary TRC-20 Implementation Module 21

3.21.1 Energy Optimization Strategy 21

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.22 Deflationary Token Model 22

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken22 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD22";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.22: Deflationary TRC-20 Implementation Module 22

3.22.1 Energy Optimization Strategy 22

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.23 Deflationary Token Model 23

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken23 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD23";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.23: Deflationary TRC-20 Implementation Module 23

3.23.1 Energy Optimization Strategy 23

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.24 Deflationary Token Model 24

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken24 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD24";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256 amount) internal {
21         require(sender != address(0), "Transfer from zero address");
22         require(recipient != address(0), "Transfer to zero address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.24: Deflationary TRC-20 Implementation Module 24

3.24.1 Energy Optimization Strategy 24

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.25 Deflationary Token Model 25

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken25 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD25";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.25: Deflationary TRC-20 Implementation Module 25

3.25.1 Energy Optimization Strategy 25

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.26 Deflationary Token Model 26

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken26 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD26";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.26: Deflationary TRC-20 Implementation Module 26

3.26.1 Energy Optimization Strategy 26

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.27 Deflationary Token Model 27

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken27 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD27";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.27: Deflationary TRC-20 Implementation Module 27

3.27.1 Energy Optimization Strategy 27

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.28 Deflationary Token Model 28

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken28 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD28";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

Listing 3.28: Deflationary TRC-20 Implementation Module 28

3.28.1 Energy Optimization Strategy 28

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.29 Deflationary Token Model 29

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken29 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD29";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

Listing 3.29: Deflationary TRC-20 Implementation Module 29

3.29.1 Energy Optimization Strategy 29

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.30 Deflationary Token Model 30

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken30 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD30";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.30: Deflationary TRC-20 Implementation Module 30

3.30.1 Energy Optimization Strategy 30

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.31 Deflationary Token Model 31

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken31 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD31";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.31: Deflationary TRC-20 Implementation Module 31

3.31.1 Energy Optimization Strategy 31

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.32 Deflationary Token Model 32

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken32 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD32";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.32: Deflationary TRC-20 Implementation Module 32

3.32.1 Energy Optimization Strategy 32

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.33 Deflationary Token Model 33

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken33 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD33";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.33: Deflationary TRC-20 Implementation Module 33

3.33.1 Energy Optimization Strategy 33

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.34 Deflationary Token Model 34

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken34 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD34";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.34: Deflationary TRC-20 Implementation Module 34

3.34.1 Energy Optimization Strategy 34

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.35 Deflationary Token Model 35

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken35 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD35";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.35: Deflationary TRC-20 Implementation Module 35

3.35.1 Energy Optimization Strategy 35

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.36 Deflationary Token Model 36

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken36 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD36";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.36: Deflationary TRC-20 Implementation Module 36

3.36.1 Energy Optimization Strategy 36

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.37 Deflationary Token Model 37

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken37 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD37";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.37: Deflationary TRC-20 Implementation Module 37

3.37.1 Energy Optimization Strategy 37

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.38 Deflationary Token Model 38

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken38 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD38";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.38: Deflationary TRC-20 Implementation Module 38

3.38.1 Energy Optimization Strategy 38

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.39 Deflationary Token Model 39

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken39 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD39";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.39: Deflationary TRC-20 Implementation Module 39

3.39.1 Energy Optimization Strategy 39

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.40 Deflationary Token Model 40

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken40 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD40";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.40: Deflationary TRC-20 Implementation Module 40

3.40.1 Energy Optimization Strategy 40

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.41 Deflationary Token Model 41

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken41 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD41";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.41: Deflationary TRC-20 Implementation Module 41

3.41.1 Energy Optimization Strategy 41

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.42 Deflationary Token Model 42

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken42 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD42";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.42: Deflationary TRC-20 Implementation Module 42

3.42.1 Energy Optimization Strategy 42

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.43 Deflationary Token Model 43

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken43 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD43";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.43: Deflationary TRC-20 Implementation Module 43

3.43.1 Energy Optimization Strategy 43

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.44 Deflationary Token Model 44

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken44 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD44";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

Listing 3.44: Deflationary TRC-20 Implementation Module 44

3.44.1 Energy Optimization Strategy 44

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.45 Deflationary Token Model 45

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken45 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD45";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.45: Deflationary TRC-20 Implementation Module 45

3.45.1 Energy Optimization Strategy 45

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.46 Deflationary Token Model 46

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken46 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD46";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.46: Deflationary TRC-20 Implementation Module 46

3.46.1 Energy Optimization Strategy 46

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.47 Deflationary Token Model 47

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken47 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD47";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.47: Deflationary TRC-20 Implementation Module 47

3.47.1 Energy Optimization Strategy 47

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.48 Deflationary Token Model 48

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken48 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD48";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.48: Deflationary TRC-20 Implementation Module 48

3.48.1 Energy Optimization Strategy 48

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.49 Deflationary Token Model 49

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken49 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD49";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public
        allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256
        amount) internal {
21         require(sender != address(0), "Transfer from zero address
        ");
22         require(recipient != address(0), "Transfer to zero
        address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external
        returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }

```

37 }

Listing 3.49: Deflationary TRC-20 Implementation Module 49

3.49.1 Energy Optimization Strategy 49

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

3.50 Deflationary Token Model 50

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract DeflationaryToken50 {
5      string public name = "WhiteSolutions Deflationary";
6      string public symbol = "WSD50";
7      uint8 public decimals = 6;
8      uint256 public totalSupply;
9
10     mapping(address => uint256) public balanceOf;
11     mapping(address => mapping(address => uint256)) public allowance;
12
13     uint256 public burnRate = 2; // 2% burn on transfer
14
15     constructor(uint256 _initialSupply) {
16         totalSupply = _initialSupply * 10 ** uint256(decimals);
17         balanceOf[msg.sender] = totalSupply;
18     }
19
20     function _transfer(address sender, address recipient, uint256 amount) internal {
21         require(sender != address(0), "Transfer from zero address");
22         require(recipient != address(0), "Transfer to zero address");
23
24         uint256 burnAmount = (amount * burnRate) / 100;
25         uint256 sendAmount = amount - burnAmount;
26
27         balanceOf[sender] -= amount;
28         balanceOf[recipient] += sendAmount;
29         totalSupply -= burnAmount;
30     }
31
32     // Transfer logic hooks into _transfer
33     function transfer(address recipient, uint256 amount) external returns (bool) {
34         _transfer(msg.sender, recipient, amount);
35         return true;
36     }
```

37 }

Listing 3.50: Deflationary TRC-20 Implementation Module 50

3.50.1 Energy Optimization Strategy 50

When deploying this module on TRON, energy delegation is crucial. Ensure you rent sufficient energy via TRON lending platforms to avoid burning TRX during high-volume transfer periods.

Chapter 4

Staking and Yield Farming Protocols

4.1 Staking Vault Boilerplate 1

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault1 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using IERC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
```

```
22         uint256 daysStaked = timeStaked / 86400;  
23         return (stakedBalances[user] * rewardRate * daysStaked) /  
24         10000;  
25     }
```

Listing 4.1: Staking Vault Contract 1

4.1.1 Yield Analysis 1

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.2 Staking Vault Boilerplate 2

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault2 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.2: Staking Vault Contract 2

4.2.1 Yield Analysis 2

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.3 Staking Vault Boilerplate 3

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault3 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.3: Staking Vault Contract 3

4.3.1 Yield Analysis 3

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.4 Staking Vault Boilerplate 4

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault4 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.4: Staking Vault Contract 4

4.4.1 Yield Analysis 4

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.5 Staking Vault Boilerplate 5

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault5 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.5: Staking Vault Contract 5

4.5.1 Yield Analysis 5

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.6 Staking Vault Boilerplate 6

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault6 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.6: Staking Vault Contract 6

4.6.1 Yield Analysis 6

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.7 Staking Vault Boilerplate 7

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault7 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.7: Staking Vault Contract 7

4.7.1 Yield Analysis 7

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.8 Staking Vault Boilerplate 8

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault8 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.8: Staking Vault Contract 8

4.8.1 Yield Analysis 8

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.9 Staking Vault Boilerplate 9

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault9 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.9: Staking Vault Contract 9

4.9.1 Yield Analysis 9

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.10 Staking Vault Boilerplate 10

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault10 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.10: Staking Vault Contract 10

4.10.1 Yield Analysis 10

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.11 Staking Vault Boilerplate 11

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault11 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.11: Staking Vault Contract 11

4.11.1 Yield Analysis 11

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.12 Staking Vault Boilerplate 12

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault12 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.12: Staking Vault Contract 12

4.12.1 Yield Analysis 12

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.13 Staking Vault Boilerplate 13

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault13 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.13: Staking Vault Contract 13

4.13.1 Yield Analysis 13

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.14 Staking Vault Boilerplate 14

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault14 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.14: Staking Vault Contract 14

4.14.1 Yield Analysis 14

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.15 Staking Vault Boilerplate 15

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault15 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.15: Staking Vault Contract 15

4.15.1 Yield Analysis 15

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.16 Staking Vault Boilerplate 16

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault16 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.16: Staking Vault Contract 16

4.16.1 Yield Analysis 16

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.17 Staking Vault Boilerplate 17

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault17 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.17: Staking Vault Contract 17

4.17.1 Yield Analysis 17

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.18 Staking Vault Boilerplate 18

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault18 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.18: Staking Vault Contract 18

4.18.1 Yield Analysis 18

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.19 Staking Vault Boilerplate 19

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault19 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.19: Staking Vault Contract 19

4.19.1 Yield Analysis 19

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.20 Staking Vault Boilerplate 20

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault20 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.20: Staking Vault Contract 20

4.20.1 Yield Analysis 20

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.21 Staking Vault Boilerplate 21

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault21 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.21: Staking Vault Contract 21

4.21.1 Yield Analysis 21

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.22 Staking Vault Boilerplate 22

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault22 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.22: Staking Vault Contract 22

4.22.1 Yield Analysis 22

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.23 Staking Vault Boilerplate 23

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault23 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.23: Staking Vault Contract 23

4.23.1 Yield Analysis 23

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.24 Staking Vault Boilerplate 24

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault24 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.24: Staking Vault Contract 24

4.24.1 Yield Analysis 24

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.25 Staking Vault Boilerplate 25

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault25 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.25: Staking Vault Contract 25

4.25.1 Yield Analysis 25

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.26 Staking Vault Boilerplate 26

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault26 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.26: Staking Vault Contract 26

4.26.1 Yield Analysis 26

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.27 Staking Vault Boilerplate 27

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault27 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.27: Staking Vault Contract 27

4.27.1 Yield Analysis 27

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.28 Staking Vault Boilerplate 28

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault28 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.28: Staking Vault Contract 28

4.28.1 Yield Analysis 28

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.29 Staking Vault Boilerplate 29

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract StakingVault29 {
5      // Staking logic variables
6      mapping(address => uint256) public stakedBalances;
7      mapping(address => uint256) public stakingTimestamp;
8
9      uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.29: Staking Vault Contract 29

4.29.1 Yield Analysis 29

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.30 Staking Vault Boilerplate 30

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault30 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.30: Staking Vault Contract 30

4.30.1 Yield Analysis 30

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.31 Staking Vault Boilerplate 31

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault31 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.31: Staking Vault Contract 31

4.31.1 Yield Analysis 31

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.32 Staking Vault Boilerplate 32

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault32 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.32: Staking Vault Contract 32

4.32.1 Yield Analysis 32

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.33 Staking Vault Boilerplate 33

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault33 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.33: Staking Vault Contract 33

4.33.1 Yield Analysis 33

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.34 Staking Vault Boilerplate 34

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault34 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.34: Staking Vault Contract 34

4.34.1 Yield Analysis 34

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.35 Staking Vault Boilerplate 35

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault35 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.35: Staking Vault Contract 35

4.35.1 Yield Analysis 35

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.36 Staking Vault Boilerplate 36

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault36 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.36: Staking Vault Contract 36

4.36.1 Yield Analysis 36

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.37 Staking Vault Boilerplate 37

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault37 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.37: Staking Vault Contract 37

4.37.1 Yield Analysis 37

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.38 Staking Vault Boilerplate 38

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault38 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.38: Staking Vault Contract 38

4.38.1 Yield Analysis 38

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.39 Staking Vault Boilerplate 39

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault39 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }

```

Listing 4.39: Staking Vault Contract 39

4.39.1 Yield Analysis 39

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

4.40 Staking Vault Boilerplate 40

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract StakingVault40 {
5     // Staking logic variables
6     mapping(address => uint256) public stakedBalances;
7     mapping(address => uint256) public stakingTimestamp;
8
9     uint256 public rewardRate = 100; // 100 tokens per day
10
11     function stake(uint256 amount) external {
12         require(amount > 0, "Cannot stake 0");
13
14         // Insert transferFrom logic here using ITRC20
15
16         stakedBalances[msg.sender] += amount;
17         stakingTimestamp[msg.sender] = block.timestamp;
18     }
19
20     function calculateReward(address user) public view returns (
21         uint256) {
22         uint256 timeStaked = block.timestamp - stakingTimestamp[
23             user];
24         uint256 daysStaked = timeStaked / 86400;
25         return (stakedBalances[user] * rewardRate * daysStaked) /
26             10000;
27     }
28 }
```

Listing 4.40: Staking Vault Contract 40

4.40.1 Yield Analysis 40

This vault template uses block timestamps for calculating reward distributions. While slightly less precise than block-height calculations, it uses significantly less CPU energy, reducing the transaction cost on the TRON network.

Chapter 5

Governance and Multi-Signature Wallets

5.1 Multi-Sig Authority Contract 1

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet1 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
```

```

24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         }));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
42         transactions[_txIndex].numConfirmations += 1;
43     }
44 }

```

Listing 5.1: Multi-Signature Implementation 1

5.1.1 Security Audit Check 1

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.2 Multi-Sig Authority Contract 2

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet2 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.2: Multi-Signature Implementation 2

5.2.1 Security Audit Check 2

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.3 Multi-Sig Authority Contract 3

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet3 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.3: Multi-Signature Implementation 3

5.3.1 Security Audit Check 3

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.4 Multi-Sig Authority Contract 4

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet4 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.4: Multi-Signature Implementation 4

5.4.1 Security Audit Check 4

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.5 Multi-Sig Authority Contract 5

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet5 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.5: Multi-Signature Implementation 5

5.5.1 Security Audit Check 5

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.6 Multi-Sig Authority Contract 6

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet6 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.6: Multi-Signature Implementation 6

5.6.1 Security Audit Check 6

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.7 Multi-Sig Authority Contract 7

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet7 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.7: Multi-Signature Implementation 7

5.7.1 Security Audit Check 7

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.8 Multi-Sig Authority Contract 8

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet8 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.8: Multi-Signature Implementation 8

5.8.1 Security Audit Check 8

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.9 Multi-Sig Authority Contract 9

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet9 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.9: Multi-Signature Implementation 9

5.9.1 Security Audit Check 9

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.10 Multi-Sig Authority Contract 10

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet10 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.10: Multi-Signature Implementation 10

5.10.1 Security Audit Check 10

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.11 Multi-Sig Authority Contract 11

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet11 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.11: Multi-Signature Implementation 11

5.11.1 Security Audit Check 11

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.12 Multi-Sig Authority Contract 12

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet12 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.12: Multi-Signature Implementation 12

5.12.1 Security Audit Check 12

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.13 Multi-Sig Authority Contract 13

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet13 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.13: Multi-Signature Implementation 13

5.13.1 Security Audit Check 13

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.14 Multi-Sig Authority Contract 14

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet14 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.14: Multi-Signature Implementation 14

5.14.1 Security Audit Check 14

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.15 Multi-Sig Authority Contract 15

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet15 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.15: Multi-Signature Implementation 15

5.15.1 Security Audit Check 15

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.16 Multi-Sig Authority Contract 16

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet16 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.16: Multi-Signature Implementation 16

5.16.1 Security Audit Check 16

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.17 Multi-Sig Authority Contract 17

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet17 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.17: Multi-Signature Implementation 17

5.17.1 Security Audit Check 17

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.18 Multi-Sig Authority Contract 18

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet18 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.18: Multi-Signature Implementation 18

5.18.1 Security Audit Check 18

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.19 Multi-Sig Authority Contract 19

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet19 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.19: Multi-Signature Implementation 19

5.19.1 Security Audit Check 19

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.20 Multi-Sig Authority Contract 20

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet20 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.20: Multi-Signature Implementation 20

5.20.1 Security Audit Check 20

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.21 Multi-Sig Authority Contract 21

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet21 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.21: Multi-Signature Implementation 21

5.21.1 Security Audit Check 21

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.22 Multi-Sig Authority Contract 22

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet22 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.22: Multi-Signature Implementation 22

5.22.1 Security Audit Check 22

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.23 Multi-Sig Authority Contract 23

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet23 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.23: Multi-Signature Implementation 23

5.23.1 Security Audit Check 23

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.24 Multi-Sig Authority Contract 24

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet24 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42         transactions[_txIndex].numConfirmations += 1;  
43     }  
44 }
```

Listing 5.24: Multi-Signature Implementation 24

5.24.1 Security Audit Check 24

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.25 Multi-Sig Authority Contract 25

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet25 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.25: Multi-Signature Implementation 25

5.25.1 Security Audit Check 25

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.26 Multi-Sig Authority Contract 26

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet26 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.26: Multi-Signature Implementation 26

5.26.1 Security Audit Check 26

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.27 Multi-Sig Authority Contract 27

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet27 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.27: Multi-Signature Implementation 27

5.27.1 Security Audit Check 27

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.28 Multi-Sig Authority Contract 28

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet28 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.28: Multi-Signature Implementation 28

5.28.1 Security Audit Check 28

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.29 Multi-Sig Authority Contract 29

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract MultiSigWallet29 {
5      address[] public owners;
6      uint public requiredApprovals;
7
8      struct Transaction {
9          address to;
10         uint value;
11         bytes data;
12         bool executed;
13         uint numConfirmations;
14     }
15
16     mapping(uint => mapping(address => bool)) public isConfirmed;
17     Transaction[] public transactions;
18
19     modifier onlyOwner() {
20         bool isOwner = false;
21         for (uint i=0; i<owners.length; i++) {
22             if(owners[i] == msg.sender) isOwner = true;
23         }
24         require(isOwner, "Not owner");
25         _;
26     }
27
28     function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29         uint txIndex = transactions.length;
30         transactions.push(Transaction({
31             to: _to,
32             value: _value,
33             data: _data,
34             executed: false,
35             numConfirmations: 0
36         })));
37     }
38
39     function confirmTransaction(uint _txIndex) public onlyOwner {
40         require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41         isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.29: Multi-Signature Implementation 29

5.29.1 Security Audit Check 29

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

5.30 Multi-Sig Authority Contract 30

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract MultiSigWallet30 {
5     address[] public owners;
6     uint public requiredApprovals;
7
8     struct Transaction {
9         address to;
10        uint value;
11        bytes data;
12        bool executed;
13        uint numConfirmations;
14    }
15
16    mapping(uint => mapping(address => bool)) public isConfirmed;
17    Transaction[] public transactions;
18
19    modifier onlyOwner() {
20        bool isOwner = false;
21        for (uint i=0; i<owners.length; i++) {
22            if(owners[i] == msg.sender) isOwner = true;
23        }
24        require(isOwner, "Not owner");
25        _;
26    }
27
28    function submitTransaction(address _to, uint _value, bytes
memory _data) public onlyOwner {
29        uint txIndex = transactions.length;
30        transactions.push(Transaction({
31            to: _to,
32            value: _value,
33            data: _data,
34            executed: false,
35            numConfirmations: 0
36        })));
37    }
38
39    function confirmTransaction(uint _txIndex) public onlyOwner {
40        require(!isConfirmed[_txIndex][msg.sender], "Tx already
confirmed");
41        isConfirmed[_txIndex][msg.sender] = true;
```

```
42     transactions[_txIndex].numConfirmations += 1;  
43 }  
44 }
```

Listing 5.30: Multi-Signature Implementation 30

5.30.1 Security Audit Check 30

Ensure ‘requiredApprovals’ is rigidly checked during initialization. It must never be greater than the length of the ‘owners’ array to prevent locked funds.

Chapter 6

Conclusion

This library serves as your foundation for building highly secure, scalable decentralized applications. Always conduct professional audits before mainnet deployment.